

Unix V6++字符设备管理

同济大学电信学院计算机系

黄炎* 陈闽中

2011 年 7 月 17 日

*黄炎师弟是 Unix V6++字符设备管理模块的设计者，这是完全不同于 Unix V6 的设计。

邓蓉 2024 年 11 月 25 日

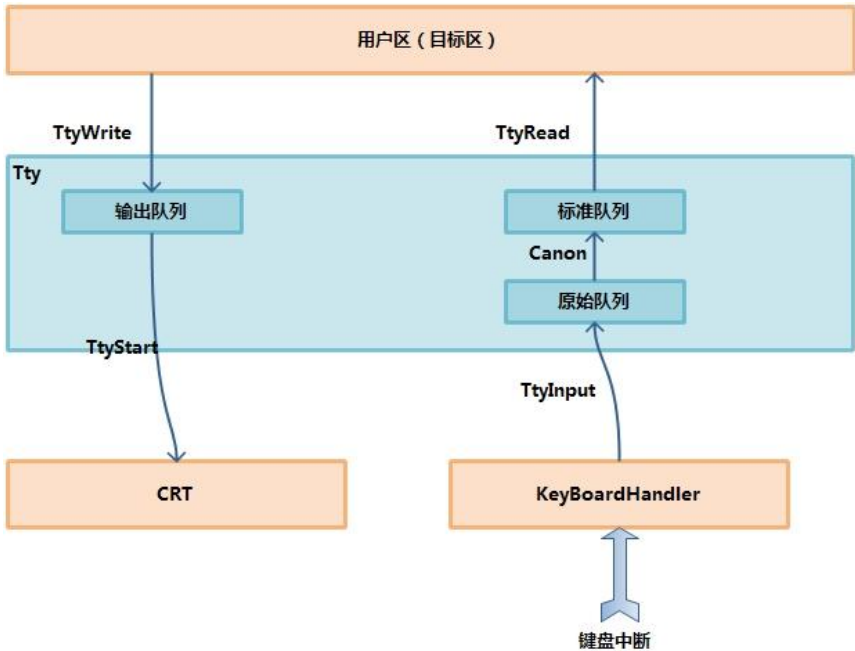
字符设备，是处理字符 输入/输出的设备。在 UnixV6++中主要涉及的字符设备如下：

- 1. 键盘，输入字符
- 2. CRT，输出字符
- 3. Tty 终端，直观上是一个键盘和一个 CRT 的输入/输出组合，用于管理单一用户的输入/输出

由于设备速度与 CPU 处理速度的差异，我们需要使用缓存技术提高字符的处理效率。主要用到的缓存结构是缓存队列，由 Tty 提供了三个缓存队列：

- 1. 原始队列，存放从 Keyboard 获得的原始输入字符
- 2. 标准队列，对原始队列中的元素进行处理（如遇到删除字符进行删除操作，遇到删行进行删行操作）后的队列
- 3. 输出队列，向 CRT 类输出的字符队列

下图展示了 Tty 与 CRT 及键盘的字符流向，及各缓存队列间的关系和主要函数名，在这里只需有个直观的印象，具体内容在相应章节中会介绍



CRT 类

CRT 类负责向屏幕的字符输出，提供基本的输出接口，将输出队列的内容输出到屏幕上

核心变量

包括：

1. `m_VideoMemory` 显存地址，前面知识我们已知这个常量的来历
2. `m_CursorX`, `m_CursorY` 记录光标位置，即下一个输出字符的位置
3. `m_Position` 在输出队列中，标记下一个输出字符
4. `m_BeginChar` 在输出队列中，标记已确认的字符的下一字符。回车操作，被认为是确认字符的操作，回车符前的所有字符将被确认，不能再被“退格键”删除，而未确认字符可以被“退格键”删除。直观上说，`m_BeginChar` 指向当前行首（由于回车键的换行），规定了“退格键”的作用范围

核心函数

核心函数的介绍将按照调用关系从底层向高层的顺序：

MoveCursor

负责移动光标到指定位置

光标的移动主要是对寄存器的读写操作，先介绍所用到的寄存器：

VGA 系统中，CRT Controller Registers 寄存器组，共 25 个寄存器，我们用到的是

1. Cursor Location High Register，索引 0Eh
2. Cursor Location Low Register，索引 0Fh

这个寄存器组包括两个端口

1. 地址端口 0x3D4
2. 数据端口 0x3D5

通过端口读写 Cursor Location 的方法代码如下：

```
IOPort::OutByte(0x3D4, 0xE);
IOPort::OutByte(0x3D5, cursorPosition >> 8);
IOPort::OutByte(0x3D4, 0xF);
IOPort::OutByte(0x3D5, cursorPosition & 0xFF);
```

如代码，先向地址端口输出寄存器索引，再向数据端口输出寄存器数据来写指定寄存器。

这里将光标位置低 8 位和高 8 位分别输出到对应寄存器，完成光标位置的设定

MoveCursor 的代码：

```

void CRT::MoveCursor(unsigned int col, unsigned int row)
{
    if ( (col < 0 || col >= CRT::COLUMNS) || (row < 0 || row >= CRT::ROWS) )
    {
        return;
    }

    /* 计算光标偏移量 */
    unsigned short cursorPosition = row * CRT::COLUMNS + col;

    /* 选择 r14 和 r15 寄存器，分别为光标位置的高 8 位和低 8 位 */
    IOPort::OutByte(CRT::VIDEO_ADDR_PORT, 14);
    IOPort::OutByte(CRT::VIDEO_DATA_PORT, cursorPosition >> 8);
    IOPort::OutByte(CRT::VIDEO_ADDR_PORT, 15);
    IOPort::OutByte(CRT::VIDEO_DATA_PORT, cursorPosition & 0xFF);
}

```

ClearScreen

清屏，代码中 0x0A00 的作用已在前面章节说明过

```

void CRT::ClearScreen()
{
    unsigned int i;

    for ( i = 0; i < COLUMNS * ROWS; i++ )
    {
        m_VideoMemory[i] = (unsigned short)' ' | 0x0A00;
    }
}

```

NextLine

换行时，进行参数调整

```
void CRT::NextLine()
{
    m_CursorX = 0;
    m_CursorY += 1;

    /* 超出最大行数 */
    if ( m_CursorY >= CRT::ROWS )
    {
        m_CursorY = 0;
        ClearScreen();
    }
    MoveCursor(m_CursorX, m_CursorY);
}
```

WriteChar

向屏幕光标当前位置输出字符，超出列限时换行

```
void CRT::WriteChar(char ch)
{
    m_VideoMemory[m_CursorY * CRT::COLUMNS + m_CursorX] = (unsigned char) ch | 0x0A00;
    m_CursorX++;

    if ( m_CursorX >= CRT::COLUMNS )
    {
        NextLine();
    }
    MoveCursor(m_CursorX, m_CursorY);
}
```

BackSpace

负责进行退格的参数修正，考虑一下的情况

1. 光标不在行首，退格将光标后退一个位置
2. 光标在行首时，退格将光标退回上一行末尾
3. 退行时，已经到顶行，重置参数

```

void CRT::BackSpace()
{
    m_CursorX--;

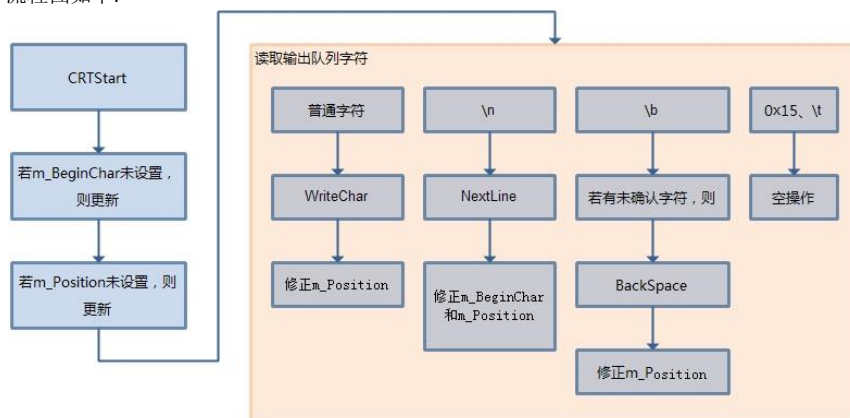
    /* 移动光标，如果要回到上一行的话 */
    if ( m_CursorX < 0 )
    {
        m_CursorX = CRT::COLUMNS - 1;
        m_CursorY--;
        if ( m_CursorY < 0 )
        {
            m_CursorY = 0;
        }
    }
}
MoveCursor(m_CursorX, m_CursorY);

/* 在光标所在位置填上空格 */
m_VideoMemory[m_CursorY * COLUMNS + m_CursorX] = ' ' | 0x0A00;
}

```

CRTStart

这是 CRT 类最核心的函数，负责输出输出队列中的内容，直到队列空
流程图如下：



代码如下：

```

void CRT::CRTStart(TTy* pTTY)
{
    char ch;
    if ( 0 == CRT::m_BeginChar)
    {
        m_BeginChar = pTTY->t_outq.CurrentChar();
    }
    if ( 0 == m_Position )
    {
        m_Position = m_BeginChar;
    }

    while ( (ch = pTTY->t_outq.GetChar()) != TTy::GET_ERROR )
    {
        switch (ch)
        {
            case '\n':
                NextLine();
                CRT::m_BeginChar = pTTY->t_outq.CurrentChar();
                m_Position = CRT::m_BeginChar;
                break;
            case 0x15:
                //del_line();
                break;
            case '\b':
                if ( m_Position != CRT::m_BeginChar )
                {
                    BackSpace();
                    m_Position--;
                }
                break;
            case '\t':
                //m_Position++;
                break;

            default: /* 在屏幕上回显普通字符 */
                WriteChar(ch);
                m_Position++;
                break;
        }
    }
}

```

这里需要解释的是参数 `m_BeginChar` 和 `m_Position` 的修正，内容如下：

1. 若 `m_BeginChar` 无值,则置为输出队列的 `CurrentChar`(实际为输出队列下一个输出字符),意义是之前所有的字符都被“确认”了
2. 若 `m_Position` 无值,置初值为 `m_BeginChar`。只在初始时有效,与上句实际意义就是 `m_Position = m_BeginChar = 输出队列的 CurrentChar`。
3. 处理“\n”时, `m_Position = m_BeginChar = 输出队列的 CurrentChar`,意义是确认之前的所有字符
4. 处理“\b”时, `m_Position != CRT::m_BeginChar` 的意义就是存在未确认字符

Keyboard 类

键盘基础知识

我们在编写键盘驱动程序时,需要了解以下两个问题:

1. 按键过程中,键盘的硬件接口提供给我们的是怎样的数据
2. 这些数据跟按键所代表的意义(字母等)有什么对应关系

首先,键盘的硬件接口提供给我们的是扫描码(ScanCode),分为 `MakeCode` 和 `BreakCode`,对应于一个键的按下和释放。举个例子,当我们按下 `A` 键时,我们将收到 `0x1E` (`MakeCode`),当我们释放 `A` 键时,我们将收到 `0x9E` (`BreakCode`)。这里的规律是 `BreakCode=MakeCode | 0x80` 当然,并不是所有键都是类似于 `A` 键的情况,有以下两种特殊的键由于特殊的原因具有一个特殊的扫描码序(而非单一的码):

1. 诸如 `Right Shift`、`Insert` 这些功能键,具有 `0xE0 + Code` 的扫描码序列
2. `Pause` 键,扫描码序为 `0xE1 0x1D 0x45` 和 `0xE1 0x9D 0xC5`

在处理键盘扫描码的过程中我们将其分为 单码、双码、三码 三种情况讨论。

每个键按下/释放时,会产生键盘中断,此时我们可以通过以下两个寄存器读取键盘的状态和数据:

1. `0x60` 数据寄存器,可以从这个寄存器读取键盘键的扫描码
2. `0x64` 状态寄存器,可以读取键盘的状态,具体用到时我们在代码中解释相应位的含义

批注 [T1]: 关于这一点,代码中有问题

核心变量

Keymap 和 Shift_Keymap

分别是 无 shift 和 shift 两种状态下,扫描码到字母的映射表,具体值如下:

```

char Keyboard::Keymap[] = {
    0,0x1b,'1','2','3','4','5','6',          /* 0x00-0x07 0, <esc>,1,2,3,4,5,6, */
    '7','8','9','0','-','=',0x8,0x9,          /* 0x08-0x0f 7,8,9,0,-,=,<backspace><tab>*/
    'q','w','e','r','t','y','u','i',          /* 0x10-0x17 qwertyui*/
    'o','p','[',']','\n','0','a','s',          /* 0x18-0x1f op[] <enter><ctrl>as */
    'd','f','g','h','j','k','l',';',          /* 0x20-0x27 dfg hjkl; */
    '\'',',',',',',',',',',',',',',',',',', /* 0x28-0x2f ``<lshift>\zdcv */
    'b','n','m',',',',',',',',',',',',',', /* 0x30-0x37 bnm,./<rshif><printscr> */
    0,' ',0,0,0,0,0,0,                      /*0x38-0x3f
<alt><space><caps><f1><f2><f3><f4><f5> */
    0,0,0,0,0,0,0,'7',                      /* 0x40-0x47 <f0><><><><><numlock><scrlock>7*/
    '8','9','-','4','5','6','+',',','1',      /* 0x48-x04f 89-456+1 */
    '2','3','0','.',',',0,0,0,0              /* 0x50-0x57 230.<><><><> */
};

char Keyboard::Shift_Keymap[] = {
    0,0x1b,'!',',','@','#','$','%','^',        /* 0x00-0x07 0, <esc>,1,2,3,4,5,6, */
    '&','*','(',')','_','+',0x8,0x9,          /* 0x08-0x0f 7,8,9,0,-,=,<backspace><tab>*/
    'q','w','e','r','t','y','u','i',          /* 0x10-0x17 qwertyui*/
    'o','p','[',']','\n','0','a','s',          /* 0x18-0x1f op[] <enter><ctrl>as */
    'd','f','g','h','j','k','l',';',          /* 0x20-0x27 dfg hjkl; */
    '\'',',',',',',',',',',',',',',',',',', /* 0x28-0x2f ``<lshift>\zdcv */
    'b','n','m','<','>','?',',','*',          /* 0x30-0x37 bnm,./<rshif><printscr> */
    0,' ',0,0,0,0,0,0,                      /*0x38-0x3f
<alt><space><caps><f1><f2><f3><f4><f5> */
    0,0,0,0,0,0,0,0,                      /* 0x40-0x47 <f0><><><><><numlock><scrlock>7*/
    0,0,'-',0,0,0,0,'+',0,                  /* 0x48-x04f 89-456+1 */
    0,0,0,0x7f/*del*/,0,0,0,0              /* 0x50-0x57 230.<><><><> */
};

```

Mode

模式变量，表示 Caps Lock、Num Lock 等状态，以及 Shift、Ctrl 等是否按下，状态常量如下：

```

/* 以下为控制键按下的状态，mode 中各比特位的定义 */
static const int M_LCTRL = 0x01;
static const int M_RCTRL = 0x02;
static const int M_LALT = 0x04;
static const int M_RALT = 0x08;
static const int M_LSHIFT = 0x10;
static const int M_RSHIFT = 0x20;
static const int M_NUMLOCK = 0x40;
static const int M_CAPSLOCK = 0x80;
static const int M_SCRLOCK = 0x100;
static const int M_DOWN_NUMLOCK = 0x200;
static const int M_DOWN_CAPSLOCK = 0x400;
static const int M_DOWN_SCRLOCK = 0x800;

```

核心函数

核心函数的介绍顺序将按照调用高层到调用底层的顺序，从键盘中断函数开始

KeyboardHandler

键盘上每个键的按下/释放，都会引发键盘中断，我们从键盘中断处理函数开始介绍

```

void Keyboard::KeyboardHandler( struct pt_regs* reg, struct pt_context* context )
{
    /******//for test
    //Diagnose::Write("key pressed !  ");
    /******

    unsigned char status = IOPort::InByte(Keyboard::STATUS_PORT);
    int limit = 10;
    static int pre_state = 0;

    while ( (status & Keyboard::DATA_BUFFER_BUSY) && limit-- )
    {
        /* 如果键盘缓存满就要读入剩下的扫描码 */
        unsigned char scancode = IOPort::InByte(Keyboard::DATA_PORT);

        /* 以下的判断过程有点像有限状态机，不过貌似有的地方没有化简 */
        if ( 0 == pre_state )
        {

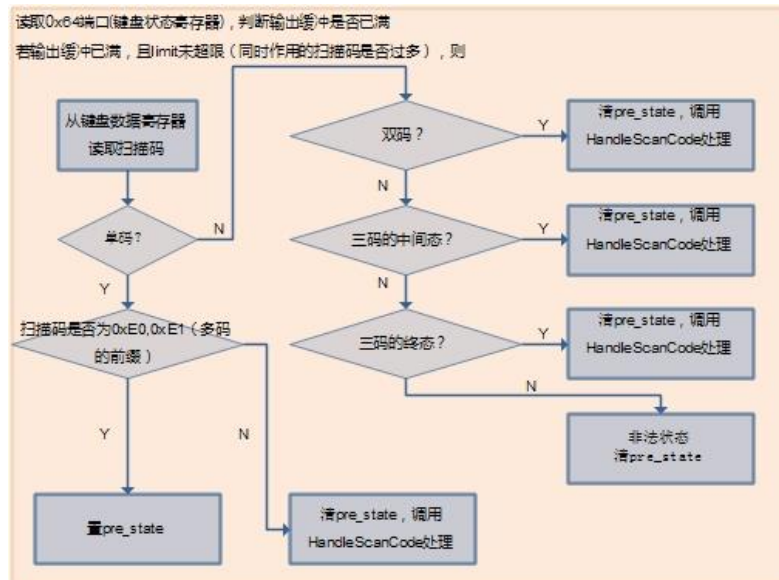
```

```

        if ( 0xE0 == scancode || 0xE1 == scancode )
        {
            /* 将状态改为 0xE0 表示还有字符没有读入 */
            /* 产生 0xE1 只有一种状态，那就是 pause 键被按下，按键按下序列
            为 0xE1,0x1d,0x45，按键断开序列为 0xE1,0x9d,0xc5 */
            pre_state = scancode;
        }
        else /* 非扩展键 */
        {
            pre_state = 0;
            Keyboard::HandleScanCode(scancode, 0);
        }
    }
    else if ( 0xE0 == pre_state )
    {
        /* 扩展键的第二个扫描码 */
        pre_state = 0;
        Keyboard::HandleScanCode(scancode, 0xE0);
    }
    else if ( 0xE1 == pre_state && ( 0x1d == scancode || 0x9d == scancode ) )
    {
        pre_state = 0x100; /* 中间状态，表示 pause 已经有两个键吻合 */
    }
    else if ( pre_state == 0x100 && 0x45 == scancode ) /* 只需要知道 pause 键什么
    时候按下 */
    {
        pre_state = 0;
        Keyboard::HandleScanCode(scancode, 0xE1);
    }
    else
    {
        pre_state = 0;
    }
    status = IOPort::InByte(Keyboard::STATUS_PORT);
}
}

```

程序流程图如下：



HandleScanCode

负责处理提取出的扫描码, 参数的形式为 `expand + scanCode` 的组合, `expand` 为双码和三码的前缀状态。代码如下:

```

void Keyboard::HandleScanCode(unsigned char scanCode, int expand)
{
    int isOK = 0;
    char ch = 0;

    if ( 0xE1 == expand )
    {
        ch = Keyboard::ScanCodeTranslate(scanCode, expand);
    }

    switch ( scanCode )
    {
    case SCAN_ALT:
        if ( 0xE0 == expand ) /* 使用扩展键, 表示是右边的 alt 按下 */
            Mode |= M_RALT;
        else
            Mode |= M_LALT;
        break;
    }
}

```

```

case SCAN_CTRL:
    if ( 0xE0 == expand )    /* 使用扩展键，表示是右边的 ctrl 按下 */
        Mode |= M_RCTRL;
    else
        Mode |= M_LCTRL;
    break;

case SCAN_LSHIFT:
    Mode |= M_LSHIFT;
    break;

case SCAN_RSHIFT:
    Mode |= M_RSHIFT;
    break;

/* 处理按键被松开的情况，清空 Mode 中按键对应的标志位 */
case SCAN_ALT + 0x80:
    if ( 0xE0 == expand )
        Mode &= ~M_RALT;
    else
        Mode &= ~M_LALT;
    break;

case SCAN_CTRL + 0x80:
    if ( 0xE0 == expand )
        Mode &= ~M_RCTRL;
    else
        Mode &= ~M_LCTRL;
    break;

case SCAN_LSHIFT + 0x80:
    Mode &= ~M_LSHIFT;
    break;

case SCAN_RSHIFT + 0x80:
    Mode &= ~M_RSHIFT;
    break;

/* 每次按下就反转改变状态位 */

/*
 * M_DOWN_NUMLOCK 宏表示 NUMLOCK 被按下，在没有按下的
 * 状态下按下 NumLock 键需要反转状态，并置 numlock 键没有松开
 */

```

```
case SCAN_NUMLOCK:
    isOK = Mode & M_DOWN_NUMLOCK;
    if ( !isOK )
    {
        Mode ^= M_NUMLOCK;
        Mode |= M_DOWN_NUMLOCK;
    }
    break;

case SCAN_CAPSLOCK:
    isOK = Mode & M_DOWN_CAPSLOCK;
    if ( !isOK )
    {
        Mode ^= M_CAPSLOCK;
        Mode |= M_DOWN_CAPSLOCK;
    }
    break;

case SCAN_SCRLOCK:
    isOK = Mode & M_DOWN_SCRLOCK;
    if ( !isOK )
    {
        Mode ^= M_SCRLOCK;
        Mode |= M_DOWN_SCRLOCK;
    }
    break;

/* 释放 NumLock, CapsLock, ScrollLock 键 */
case SCAN_NUMLOCK + 0x80:
    Mode &= ~M_DOWN_NUMLOCK;
    break;

case SCAN_CAPSLOCK + 0x80:
    Mode &= ~M_DOWN_CAPSLOCK;
    break;

case SCAN_SCRLOCK + 0x80:
    Mode &= ~M_DOWN_SCRLOCK;
    break;

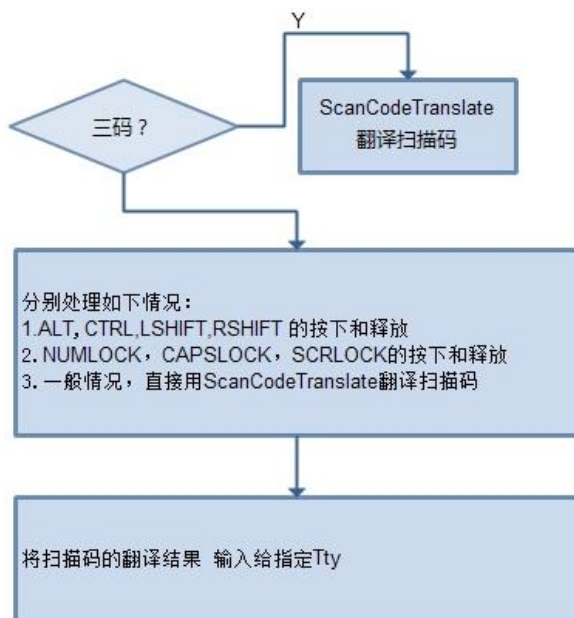
default:
    ch = Keyboard::ScanCodeTranslate(scanCode, expand);
    break;
}
```

```

    if ( 0 != ch )
    {
        TTy* pTTY =
Kernel::Instance().GetDeviceManager().GetCharDevice(DeviceManager::TTYDEV).m_TTy;
        if ( NULL != pTTY )
        {
            pTTY->TTYInput(ch);
        }
    }
}

```

流程图如下：



显然，对于功能键的处理，主要是修改 Mode。对于一般键的翻译，交给了 ScanCodeTranslate 函数

ScanCodeTranslate

负责对一般按键、Pause 键、小键盘键 的扫描码进行翻译。
由以上我们知道，其他功能键已由 HandleScanCode 处理。

```

char Keyboard::ScanCodeTranslate(unsigned char scanCode, int expand)
{
    char ch = 0;
    bool bReverse = false;

    if ( 0xE1 == expand )    /* Pause Key */
    {
        ch = 0x05;    /* Pause ASCII */
    }
    else if ( scanCode < 0x45 )    /* 非小键盘和控制键 */
    {
        /* 根据扫描码映射表找到对应按键的 ASCII 码 */
        if ( (Mode & M_LSHIFT) || (Mode & M_RSHIFT) )
        {
            ch = Shift_Keymap[scanCode];
        }
        else
        {
            ch = Keymap[scanCode];
        }

        if ( ch >= 'a' && ch <= 'z' )
        {
            /* 是小写字符而且已经 Capslock 了，那么转换成大写字符 */
            bReverse = ( (Mode & M_CAPSLOCK) ? 1 : 0 ) ^ ( (Mode & M_LSHIFT) || (Mode
& M_RSHIFT) );

            if ( (Mode & M_LCTRL) || (Mode & M_RCTRL) )    /* 按下 ctrl 进行转意 */
            {
                ch -= 'a';
                ch++;    /* 转义从 0x1 开始*/
            }
            else if ( bReverse )
            {
                ch += 'A' - 'a';
            }
        }
    }
}

```

```

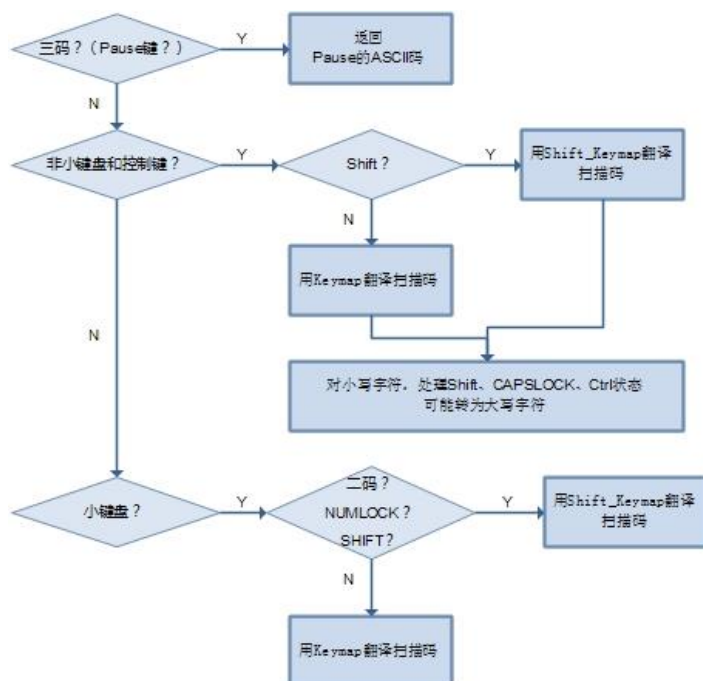
else if ( scanCode < 0x58 )
{
    bReverse = ( (Mode & M_NUMLOCK) ? 1 : 0 ) ^ ( (Mode & M_LSHIFT) || (Mode & M_RSHIFT) );

    if ( 0xE0 == expand )
    {
        ch = Shift_Keymap[scanCode];
    }
    else if ( bReverse )
    {
        ch = Keymap[scanCode];
    }
    else
    {
        ch = Shift_Keymap[scanCode];
    }
}

return ch;
}

```

流程图如下：



Tty 类

综述

Tty 类主要负责提供终端的基本功能，包括输入功能和输出功能。

其中最主要的结构是 Tty 类的三个队列：

4. 原始队列，存放从 Keyboard 获得的原始输入字符
5. 标准队列，对原始队列中的元素进行处理（如遇到删除字符进行删除操作，遇到删行进行删行操作）后的队列
6. 输出队列，向 CRT 类输出的字符队列

由函数控制的元素在各个队列中的转移过程如下：

1. 键盘按键引发 键盘中断，由 Keyboard 类的 KeyboardHandler 处理，其中调用 HandleScanCode 解析扫描码，并将解析的结果字符提交给 Tty 的 TtyInput 函数处理。
2. TtyInput 负责将字符送入原始队列。回显时，TtyInput 调用 TtyOutput 回显字符
3. TtyOutput 负责将字符送入输出队列，在 TtyStart 被调用时（Tty 设备开始工作时），输出队列的内容交由 Crt 类输出在屏幕上
4. TTRead 函数的功能是 输入到 Tty 标准队列的字符输出到目标区（由用户指定）。其中调用到 Canon 函数将原始队列的字符进行规范化，并交给标准队列
5. TTWrite 函数的功能是 将目标区的字符 输出到 Tty 的输出队列

由以上，可以完成基本的终端 I/O 功能

核心变量

三个队列

```
TTY_Queue t_rawq; /* 原始输入字符缓存队列 */
TTY_Queue t_canq; /* 标准输入字符缓存队列 */
TTY_Queue t_outq; /* 输出字符缓存队列 */
```

状态字 t_flags

状态常量包括：

```
/* modes (t_flags 设置) */
static const int HUPCL = 0x1;
static const int XTABS = 0x2;
static const int LCASE = 0x4;
static const int ECHO = 0x8;
static const int CRMOD = 0x10;
static const int RAW = 0x20;
```

其中有一些是 UnixV6 遗留下来的，已经用不到了，对于用到的状态常量进行解释：

1. LCASE 小写终端
2. ECHO 回显终端，对输入的字符回显
3. CRMOD 使用 CR 风格的换行 “\r\n”
4. RAW 原始终端，对所有的输入保持原样，不操作

核心函数

PassC 和 CPass

PassC 的功能 是将字符送入目标区

CPass 的功能 是从目标区读入字符

目标区属性 由 User.u_IOParm 确定

```
int TTy::PassC(char ch)
{
    User& u = Kernel::Instance().GetUser();

    /* 将字符送入用户目标区 */
    if ( u.u_IOParm.m_Count > 0 )
    {
        *(u.u_IOParm.m_Base++) = ch;
        //u.u_IOParm.m_Offset++;
        u.u_IOParm.m_Count--;
        return 0;
    }
    return -1;
}

char TTy::CPass()
{
    char ch;
    User& u = Kernel::Instance().GetUser();
```

```

ch = *(u.u_IOParam.m_Base++);
if ( u.u_IOParam.m_Count > 0 )
{
    u.u_IOParam.m_Count--;
    //u.u_IOParam.m_Offset++;
    return ch;
}
else
{
    return -1;
}
}

```

TtyInput

向终端输入一个字符。会将字符送入原始队列中。代码如下：

```

void TTy::TtyInput(char ch)
{
    if ( (ch &= 0xFF) == '\r' && (this->t_flags & TTy::CRMOD) )
    {
        ch = '\n';
    }

    /* 如果是小写终端 */
    if ( (this->t_flags & TTy::LCASE) && ch >= 'A' && ch <= 'Z' )
    {
        ch += 'a' - 'A';
    }

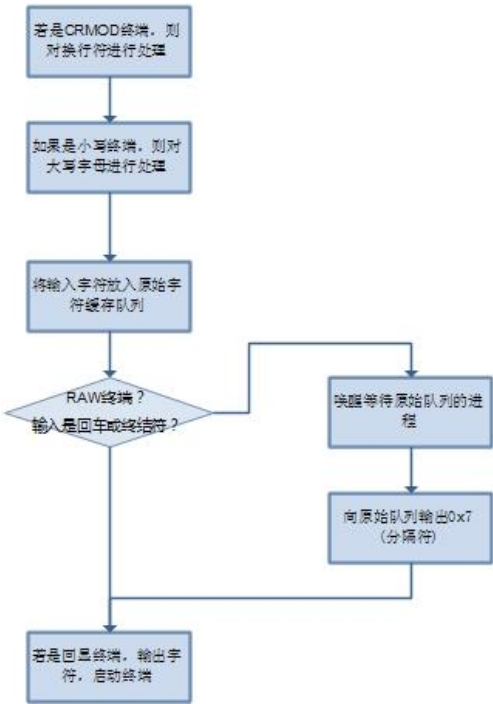
    /* 将输入字符放入原始字符缓存队列 */
    this->t_rawq.PutChar(ch);

    if ( this->t_flags & TTy::RAW || ch == '\n' || ch == TTy::CEOT )
    {
        Kernel::Instance().GetProcessManager().WakeUpAll((unsigned long)&this->t_rawq);
        this->t_rawq.PutChar(0x7);
        this->t_delct++;
    }
}

```

```
if ( this->t_flags & TTy::ECHO )
{
    this->TTYOutput(ch);
    this->TTStart();
}
}
```

流程图如下：



TtyOutput

向终端输出一个字符。字符将送入输出队列。代码如下：

```

void TTy::TTyOutput(char ch)
{
    /* 如果输入字符为文件结束符，并且终端工作在原始方式下，则返回 */
    if ( (ch &= 0xFF) == TTy::CEOT && (this->t_flags & TTy::RAW) == 0 )
    {
        return;
    }

    if ( '\n' == ch && (this->t_flags & TTy::CRMOD) )
    {
        this->TTyOutput('\r');
    }

    /* 将字符放入输出字符缓存队列中 */
    if (ch)
    {
        this->t_outq.PutChar(ch);
    }
}

```

流程图如下：



TTStart

启动终端。在这里就是启动 CRT 进行输出操作。

```

void TTy::TTStart()
{
    CRT::CRTStart(this);
}

```

Canon

对原始队列中的字符进行处理，处理 **BackSpace** 和 删行，处理到下一个分隔符，并将结果送入标准队列。

```
int TTy::Canon()
{
    char* pChar;
    char ch;
    User& u = Kernel::Instance().GetUser();

    X86Assembly::CLI();
    while ( this->t_delct == 0 )
    {
        if ( (this->t_state & TTy::CARR_ON) == 0 )
        {
            /* 设备没有打开，返回 */
            return 0;
        }
        u.u_procp->Sleep((unsigned long)&this->t_rawq, ProcessManager::TTIPRI);
    }
    X86Assembly::STI();

loop:
    pChar = &Canonb[2];
    while ( (ch = this->t_rawq.GetChar()) >= 0 )
    {
        if ( 0x7 == ch ) /* 0x07 是每个输入串的分隔符 */
        {
            this->t_delct--;
            break;
        }

        if ( (this->t_flags & TTy::RAW) == 0 ) /* 如果不是 raw 模式，需要在遇到转义
        字符时作相应的变换 */
        {
            if ( pChar[-1] != '\\' )
            {
                if ( ch == this->t_erase ) /* 如果是 backspace 键 */
                {
                    if ( pChar > &Canonb[2] ) /* 直接删除前面的字符 */
                    {
                        pChar--;
                    }
                }
            }
        }
    }
}
```

```

        continue;
    }

    if ( ch == this->t_kill )    /* 是删除一行 */
    {
        /* 从行首重新开始，这里使用 loop 虽然破坏了程序结构，但的确
        是简化了许多操作，v6 原来就是这样写的 */
        goto loop;
    }

    if ( ch == TTy::CEOT )    /* CEOT == 0x4 */
    {
        /* 遇到文件结束符并不作任何操作，这个是在将输入输出转向时
        将文件当作终端读写时为了保持一致 */
        continue;
    }
}
else
{
    /* 暂时不处理 raw 模式下的情况 */
}

*pChar++ = ch;    /* 普通字符直接放入工作缓存 */
if ( pChar >= Canonb + TTy::CANBSIZ )
{
    break;
}
}

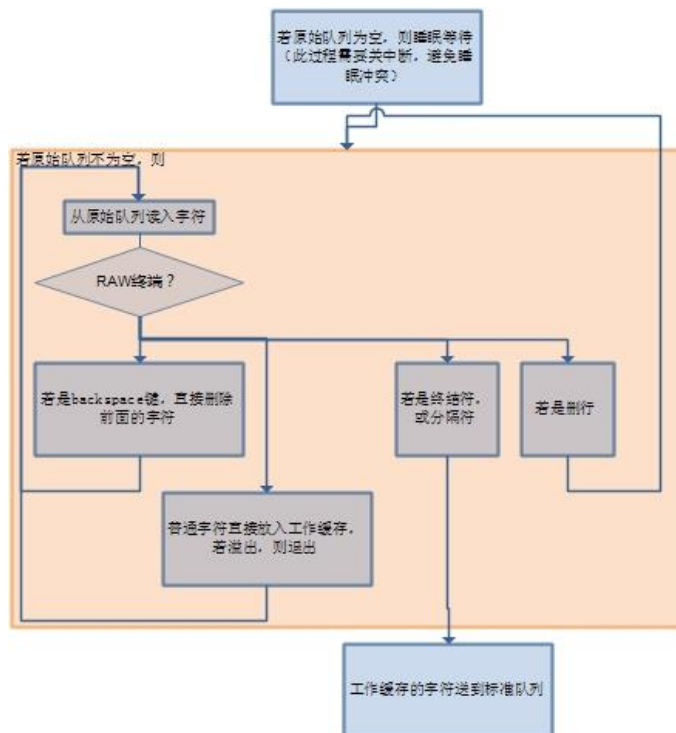
char* pEnd = pChar;
pChar = &Canonb[2];

/* 将 cook 过的字符送到行规则缓存，供 ttread 读取 */
while ( pChar < pEnd )
{
    this->t_canq.PutChar(*pChar++);
}

return 1;
}

```

流程图如下：



TTYRead

TTYRead 很简单，就是将标准队列中的元素送到指定目标区

```
void TTY::TTYRead()
{
    /* 设备没有开始工作，返回 */
    if ( (this->t_state & TTY::CARR_ON) == 0 )
    {
        return;
    }

    if ( this->t_canq.CharNum() || this->Canon() )
    {
        while ( this->t_canq.CharNum() && (this->PassC(this->t_canq.GetChar()) >= 0) );
    }
}
```

TTWrite

TTWrite 也很简单，将目标区的字符送入输出队列，在输出队列满时，启动 TTy 进行屏幕输出

```
void TTy::TTWrite()
{
    /*
     * 因为现在的输出设备是内存，其相应速度相当快，所以
     * 不需要在结束回显后发送中断，这样的代价非常大。这和
     * 原来 unix v6 的有些不同。本来可以在输出字符的过程中关
     * 中断来防止用户输入但不能被正常删除的 bug,但因为这样
     * 可能会导致时钟中断相应的延迟，这个问题更加严重，因
     * 次没有这样做。
     */
    char ch;

    /* 设备没有开始工作，返回 */
    if ( (this->t_state & TTy::CARR_ON) == 0 )
    {
        return;
    }

    while ( (ch = CPass()) > 0 )
    {
        /*输出队列中超过规定字符数，需要赶快显示 */
        if ( this->t_outq.CharNum() > TTy::TTHIWAT)
        {
            this->TTStart();
            /* 重新设置 BeginChar 指向输出字符缓存队列中，未确认部分的起始处。
             * 目的在于不允许 Backspace 键删除写在标准输出上的内容，譬如命令提示符之类。
             */
            CRT::m_BeginChar = this->t_outq.CurrentChar();
        }
        this->TTYOutput(ch);
    }
    this->TTStart();
    CRT::m_BeginChar = this->t_outq.CurrentChar();
    /* 重设 BeginChar 为了防止错误删除打印的字符，这里需要清空显示缓存，但会造成前面输入回显
     * 的字符在被删除时并不能被删掉，但是在实际中已经被删除了。
     */
}
```

FlushTTY

清空 Tty 涉及的所有队列，在此不列出代码